



SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines

Ke Cheng^{*†‡}
Nanjing University
Nanjing, China
ketonmi@outlook.com

Zhi Wang[‡]
Ant Group
Hangzhou, China
wangchun.wz@antgroup.com

Wen Hu
Ant Group
Hangzhou, China
huwen.hu@antgroup.com

Tiannuo Yang
Nankai University
Tianjin, China
yangtn@njbj.nankai.edu.cn

Jianguo Li[§]
Ant Group
Hangzhou, China
lijg.zero@antgroup.com

Sheng Zhang^{*§}
Nanjing University
Nanjing, China
sheng@nju.edu.cn

Abstract

As large language models (LLMs) are gaining increasing popularity across a wide range of web applications, it is of great importance to optimize service-level objectives (SLOs) for LLM inference services to enhance user satisfaction and improve the competitiveness of cloud vendors. In this paper, we observe that adjusting the parameters of LLM inference engines can improve service performance, and the optimal parameter configurations of different services are different. Therefore, we propose SCOOT, an automatic performance tuning system to optimize SLOs for each LLM inference service by tuning the parameters of the inference engine. SCOOT jointly exploits single-objective and multiple-objective Bayesian optimization (BO) techniques to handle various optimization objectives via exploration and exploitation. Moreover, SCOOT prunes the search space with known constraints and adopts a random forest to learn hidden constraints during the tuning process to mitigate invalid exploration. To improve the tuning efficiency, SCOOT utilizes the parallel suggestion to accelerate the tuning process. Extensive experiments demonstrate that SCOOT considerably outperforms existing tuning techniques in SLO optimization while greatly improving the tuning efficiency. Moreover, SCOOT is universally applicable to various LLM inference engines including vLLM and TensorRT-LLM. Currently, SCOOT has already been implemented in the production environment at Ant Group.

CCS Concepts

• **Computing methodologies** → **Parallel computing methodologies; Machine learning; Natural language generation.**

^{*}Ke Cheng and Sheng Zhang are with the State Key Laboratory for Novel Software Technology at Nanjing University.

[†]Work done during an internship at Ant Group.

[‡]Ke Cheng and Zhi Wang contribute equally to this work.

[§]Jianguo Li and Sheng Zhang are corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '25, April 28–May 2, 2025, Sydney, NSW, Australia.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1274-6/25/04

<https://doi.org/10.1145/3696410.3714930>

Keywords

Service-Level Objective, LLM Inference Engine, Performance Tuning, Bayesian Optimization, Cloud Computing

ACM Reference Format:

Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. 2025. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*, April 28–May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3696410.3714930>

1 Introduction

With the impressive capabilities demonstrated by large language models (LLMs) across various web applications [1], cloud vendors such as Alibaba Cloud and AWS have started offering services of LLM deployment and inference [2, 3]. Customers (e.g., developers and providers of web applications) can deploy specified LLMs on their exclusive computing resources. Through the application programming interface (API) provided by the cloud vendor, customers' requests can be continuously served by the deployed LLMs.

These LLM inference services run LLM instances with advanced inference engines such as vLLM [4] and TensorRT-LLM [5], which are equipped with cutting-edge technologies, such as continuous batching [6], paged attention [7], and chunked prefill [8]. These techniques can accelerate inference and improve throughput, thus delivering a high service performance to customers.

To ensure service performance, customers always agree on service level objectives (SLOs) with cloud vendors. SLOs are defined as a series of performance metric constraints, such as requiring that the 95th percentile latency of requests be less than 1 second. If cloud vendors violate these SLOs, they not only have to compensate customers but also face reputation damage. Therefore, appropriately setting SLOs is critical for cloud vendors [9]. They should optimize SLOs (e.g., guarantee a lower tail latency) to improve customer satisfaction while ensuring that there won't be SLO violations.

Typically, cloud vendors stress test services with high request rates and set the performance metrics achieved under the stress testing as SLOs. This ensures that SLOs won't be violated even under heavy workload scenarios. By improving the service performance under stress testing, cloud vendors can deliver better SLOs to customers. In this paper, we observe that adjusting the parameters of LLM inference engines has great potential to improve service performance, and the optimal parameter configurations of

various LLM inference services are different. Therefore, we stand in the shoes of cloud vendors and optimize SLOs for LLM inference services by tuning inference engine parameters under stress testing.

Numerous performance tuning methods have been proposed and applied across various fields [10], but they all fall short of both efficiency and optimality for tuning LLM inference engines. Methods like random sampling and meta-heuristic algorithms, such as Monte Carlo sampling [11] and genetic algorithms [12] lack efficiency as they fail to fully utilize historical information. Besides, heuristic searches rely on expert knowledge and elaborate pre-profiling to model relationships between parameters and performance. Nevertheless, with the rapid evolution of technologies in LLM inference engines, the parameters' numbers and ranges are frequently updated, which makes the modeled parameter-performance relationship outdated and the designed heuristic search methods inapplicable. Additionally, learning-based approaches such as reinforcement learning (RL) [13] and Bayesian optimization (BO) [14] have also been widely exploited in performance tuning. They can effectively leverage historical information and tune parameters automatically without prior knowledge. However, RL requires a time-consuming training process, while existing BO solutions fail to address the following three challenges.

Challenge 1: Various Optimization Objectives. Customers seek to enhance different performance metrics depending on their application requirements, such as improving request throughput for the periodically invoked offline recommendation application, reducing request tail latency for the online classification application, and minimizing time-to-first-token (TTFT) and time-per-output-token (TPOT) simultaneously for interactive applications such as chatbot, requiring the tuner to have the capability of handling both single-objective and multi-objective optimization problems.

Challenge 2: Complex Known and Hidden Constraints. Some parameters of inference engines depend on the settings of other parameters, thus causing constraints on the search space. For example, for vLLM, `max-num-batched-tokens` must be greater than or equal to `max-num-seqs`. We refer to these as known constraints, which can be provided to the tuner ahead of time. Besides, given a specific service, certain parameter combinations can lead to inference engine crashes during stress testing. In this paper, these infeasible parameter combinations are referred to as hidden constraints. Different inference services have different hidden constraints that are initially unknown and must be learned throughout the tuning process. Specifically, for some certain services, vLLM often crashes due to a timeout error during the stress testing when `scheduler-delay-factor` is set to a large value.

Challenge 3: High Evaluation Overhead. Learning-based tuners always learn the correlation between the service performance and the engine's parameter settings by evaluating various parameter configurations. As stress testing an inference service takes about 5 to 10 minutes, even with only 30 evaluations for tuning, the total time spent tuning a single service can range from 2.5 to 5 hours. Since the increasing popularity of LLMs leads to the deployment of a large number of LLM inference services, the cumulative time required for tuning these services is prohibitive.

To tackle these challenges, we propose SCOOT, a ServiCe-level Objective Oriented performance Tuning system, which automatically tune parameters of LLM inference engines to optimize SLOs

Table 1: vLLM PARAMETERS TO TUNE.

Configuration Parameter	Type	Range
<code>tensor-parallel</code>	Integer	[1, #GPUs]
<code>max-num-seqs</code>	Integer	[64, 8192]
<code>max-num-batched-tokens</code>	Integer	[64, 8192]
<code>block-size</code>	Enumeration	{8, 16, 32}
<code>scheduler-delay-factor</code>	Float	[0, 2]
<code>enable-chunked-prefill</code>	Boolean	{True, False}
<code>enable-prefix-caching</code>	Boolean	{True, False}
<code>disable-custom-all-reduce</code>	Boolean	{True, False}
<code>use-v2-block-manager</code>	Boolean	{True, False}

for LLM inference services. We first propose a general formulation of the inference engine tuning problem to accommodate various optimization objectives and complex constraints, and SCOOT can resolve the problem with BO, where single-objective BO (SOBO) and multi-objective BO (MOBO) [15–17] are respectively employed to search optimized parameter configurations for single-objective and multi-objective optimization scenarios, thus addressing challenge 1. Since constraint violations result in invalid observations caused by engine crashes, which greatly hurts the tuning efficiency, SCOOT prunes the search space with known constraints and exploits a random forest to learn hidden constraints during the tuning process, thus mitigating challenge 2. To resolve challenge 3, SCOOT employs the parallel suggestion technique to recommend multiple parameter configurations each time for simultaneous evaluation, thus fully utilizing idle computing resources to speed up tuning. SCOOT can support various LLM inference engines including vLLM and TensorRT-LLM. Nowadays, SCOOT is in use at Ant Group.

We conduct extensive experiments with various LLMs and different types and numbers of GPUs under request traces collected from various LLM-based web applications at Ant Group. The results show that SCOOT can speed up the tuning process and significantly optimize SLOs, improving the request throughput by up to 68.3%, reducing the request tail latency by up to 40.6%, and reducing the TTFT and TPOT by up to 99.8% and 61.0%, respectively, compared to the default parameter configuration and existing tuning methods. SCOOT is open-sourced at <https://github.com/Ketonmi/SCOOT>. The main contribution of this paper is summarized as follows.

- To the best of our knowledge, this is the first study that introduces performance tuning into the field of LLM serving, and we uncover the significance of tuning LLM inference engines with real-world request traces.
- We propose a general formulation of the inference engine tuning problem that accommodates various optimization objectives and constraints, and we design SCOOT to solve the problem by intelligently searching optimized parameter configurations with BO.
- Random forest regression is employed by SCOOT to learn hidden constraints during the tuning process to avoid invalid explorations, while the parallel suggestion technique is adopted to significantly improve the tuning efficiency using additional computing resources.
- Extensive experiments are conducted to confirm the superiority of SCOOT in terms of both the optimality and efficiency for tuning LLM inference engines under various LLMs, computing resources, and request traces.

Table 2: VARIOUS SERVICE CHARACTERISTICS.

Service	Application	GPU Type	GPU Number	LLM
A	SQL	A100	2	LLAMA2-13B
B	BOT	A100	2	LLAMA2-13B
C	BOT	A10	2	LLAMA2-13B
D	BOT	A10	4	LLAMA2-13B
E	BOT	A10	4	LLAMA2-7B

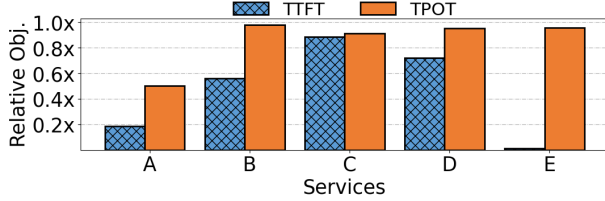
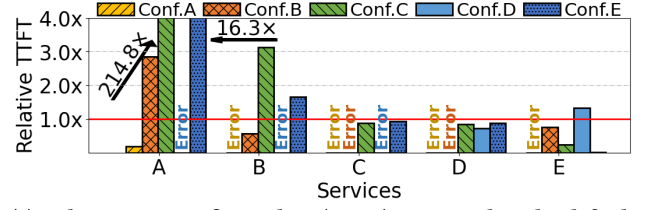


Figure 1: Optimal TTFT and TPOT for various services. The TTFT and TPOT shown are relative values compared to those of the default parameter configuration. The lower, the better.

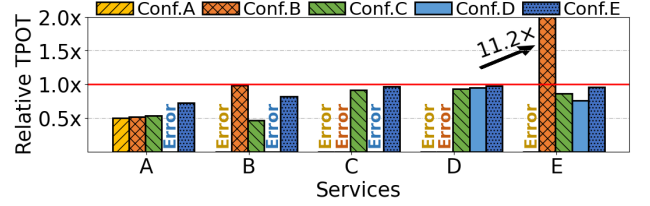
2 Background and Motivation

Background: parameters of the LLM inference engine. To provide flexibility of use, inference engines expose many parameters. For vLLM, these parameters include boolean variables such as `enable-chunked-prefill` that can enable the chunked prefill technique, integer and float variables such as `max-num-seqs` and `scheduler-delay-factor` that can change the request scheduling strategy, and enumeration variables such as `block-size` that can change the memory allocation policy. In this paper, we focus on tuning parameters that do not affect the accuracy of LLMs. Therefore, we do not consider parameters related to model compression, such as quantization. Besides, although speculative decoding has been theoretically proven not to hurt LLM accuracy [18], it still affects the LLMs’ generation results, making it inapplicable for certain applications. Hence, we also do not tune parameters related to speculative decoding. Due to the open-source nature, ease of use, and popularity of vLLM, it is primarily utilized as the inference engine in this paper, and the vLLM parameters to be tuned are listed in Table 1, which constructs a huge search space of over one hundred billion configuration points.

Motivation: parameter adjustment of the LLM inference engine can enhance performance for LLM inference services. We conduct experiments to find the optimal parameter configurations of vLLM with grid searches for five services of various LLMs deployed on different numbers and types of GPUs under request traces of various applications. The service characteristics in the experiments are shown in Table 2, where A10 and A100 separately represent the NVIDIA A10 24G GPU and NVIDIA A100 80G GPU, and SQL and BOT represent the request traces of text-to-SQL and chatbot applications at Ant Group. Other experimental settings are the same as those in Section 5. Figure 1 presents the relative TTFT and TPOT performed by optimal parameter configurations for these services. We can observe that TTFT and TPOT can be reduced by up to 98.9% and 49.9% compared to the default configuration, respectively, which confirms the significance of performance tuning for LLM inference engines.



(a) Relative time-to-first-token (TTFT) compared to the default configuration. The lower, the better.



(b) Relative time-per-output-token (TPOT) compared to the default configuration. The lower, the better.

Figure 2: TTFT and TPOT for applying optimal parameter configurations of different services to other services. Conf. X represents the optimal parameter configuration for the service $X \in \{A, B, C, D, E\}$. The red lines (1.0 $\times$) in Sub-figures (a) and (b) indicate the TTFT and TPOT under the default parameter configuration of the inference engine, respectively.

Motivation: different inference services’ optimal parameter configurations are different. We conduct experiments to apply the optimal parameter configuration of a service to other services. Figure 2 presents the experimental results. We can observe that the performance exhibited by the optimal parameter configuration for a service may perform poorly or even violate hidden constraints and cause errors for other services. Hence, there is no one “best practice” configuration that works best in all scenarios, and it is essential to conduct performance tuning for each inference service.

3 Problem Formulation

Given an inference service, the goal of performance tuning for the LLM inference engine is to find the optimal or near-optimal parameter configuration that maximizes the objective and satisfies known and hidden constraints. As we mentioned before, different customers want to optimize different performance metrics. Therefore, we provide a generalized problem formulation that supports various optimization objectives and complex constraints.

We define the configuration search space $\Lambda = \Lambda_1 \times \Lambda_2 \times \dots \times \Lambda_n$, where Λ_i represent the range of the i th parameter to tune. Vector $\mathbf{x} \in \Lambda$ is utilized to denote a specific parameter configuration, where the i th element $\mathbf{x}[i] \in \Lambda_i$ corresponds to the value of the i th parameter. Moreover, we respectively use $T(\mathbf{x})$, $L(\mathbf{x})$, $\Phi(\mathbf{x})$, and $\Theta(\mathbf{x})$ to represent the functions of request throughput, tail latency, average TTFT, and average TPOT for the inference service under configuration $\mathbf{x}$. We leverage $C = \{c_1, c_2, \dots, c_m\}$ to symbolize the set of known constraints, where c_i is the i th known constraint which restricts the relationship between elements of $\mathbf{x}$. For example, a known constraint can be expressed as “ $\mathbf{x}[1] < \mathbf{x}[3]$ if $\mathbf{x}[4]$ is False”. Moreover, we utilize $POF(\mathbf{x})$ to represent $\mathbf{x}$ ’s probability of feasibility (POF) that $\mathbf{x}$ won’t violate hidden constraints. Therefore, the tuning problem can be formulated as follows.

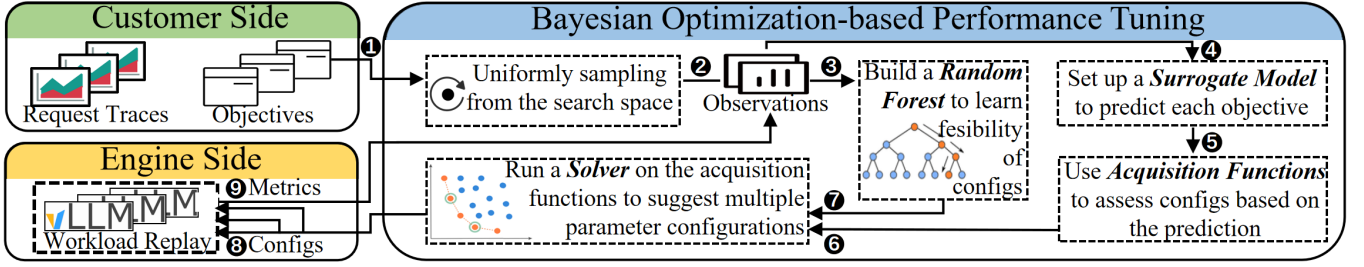


Figure 3: SCOOT workflow. SCOOT leverages BO to find optimized parameter configurations via exploration and exploitation.

$$\mathbb{P} : \max_{\mathbf{x} \in \Lambda} \lambda_t \cdot T(\mathbf{x}), \lambda_l \cdot L(\mathbf{x}), \lambda_\phi \cdot \Phi(\mathbf{x}), \lambda_\theta \cdot \Theta(\mathbf{x}) \quad (1)$$

s. t.

$$c_i, \forall c_i \in C, \quad (2)$$

$$POF(\mathbf{x}) \geq \Delta, \quad (3)$$

where Eq. (2) and Eq. (3) respectively denotes the known and hidden constraints. Δ is the POF threshold used to avoid violations of hidden constraints. $\lambda_t \in \{0, 1\}$ and $\lambda_l, \lambda_\phi, \lambda_\theta \in \{0, -1\}$ are utilized to control the optimization objectives. If $(\lambda_t, \lambda_l, \lambda_\phi, \lambda_\theta) = (1, 0, 0, 0)$, the optimization objective is to only maximize the request throughput, which is applicable to periodically invoked applications such as offline recommendation. If $(\lambda_t, \lambda_l, \lambda_\phi, \lambda_\theta) = (0, -1, 0, 0)$, the optimization objective is to only minimize the tail latency, which is appropriate for non-interactive online applications such as classification. If $(\lambda_t, \lambda_l, \lambda_\phi, \lambda_\theta) = (0, 0, -1, -1)$, the optimization objective is to simultaneously minimize the TTFT and TPOT, which is suitable for interactive applications.

4 SCOOT: Solution Description

We present SCOOT's design, including its use of SOBO and MOBO for solving $\mathbb{P}$, its innovative features of parallel suggestion and random forest-based POF learning, and the SLO robustness assurance.

4.1 SCOOT Workflow

The workflow of SCOOT is depicted in Fig. 3, which consists of nine key steps to find optimized parameter configurations. Customers define their optimization objectives, and SCOOT ① gathers their request traces that include both input text of requests and output text generated by the LLM. Then, SCOOT leverages Sobol sequence-based Quasi-Monte Carlo [19] to uniformly ② sample configurations across the search space, where the number of samples matches the search space's dimensionality. Then, SCOOT runs the inference engine with each sampled configuration and stress tests the inference service to obtain initial observations. These observations of configuration-performance pairs are leveraged to ③ build a random forest and ④ construct a surrogate model to learn the $POF(\cdot)$ and predict the probability distribution of each optimization objective, respectively. Subsequently, acquisition functions are exploited to ⑤ assess configurations according to the predicted results. Based on the assessment, a solver ⑥ suggests multiple configurations in parallel while adhering to the known constraints and ⑦ ensuring hidden constraints using $POF(\cdot)$ learned by the random forest. Lastly, the inference engine ⑧ is started with each suggested parameter configuration, and the stress testing is conducted to ⑨

obtain new observations to refine the random forest and the surrogate model. Steps ③~⑨ run iteratively, and the tuning process stops until the number of observations reaches a given threshold.

In the workflow, the LLM, GPU number, and GPU type used for tuning are the same as the inference service owned by the customer.

4.2 Bayesian Optimization-based Solution

BO is a theoretically grounded method for finding the optimum of black-box functions. It can explore the complex multi-dimensional search space efficiently and intelligently [14], which is suitable for solving problems with expensive evaluation overhead. BO leverages a surrogate model to approximate the objective function and use acquisition functions to assess configuration points for suggesting.

SCOOT leverages SOBO to maximize throughput and minimize tail latency for non-interactive offline and online applications, respectively. For interactive applications, it exploits MOBO to minimize TTFT and TPOT simultaneously by finding a set of parameter configurations representing the Pareto frontier that denotes the optimal trade-offs between TTFT and TPOT. We do not linearly combine TTFT and TPOT as a single optimization objective and solve it with SOBO because TTFT can be hundreds or even thousands of times larger than TPOT. Hence, it is difficult to assign weights to TTFT and TPOT to make a good trade-off. Besides, TTFT and TPOT are always two conflicting optimization objectives as illustrated in Appendix A.1.

4.3 Surrogate Model

The surrogate model of BO predicts the objective function $f(\mathbf{x})$ based on observations. It models $f(\mathbf{x})$ for a given configuration $\mathbf{x}$ as a random variable and predicts its probability distribution. In the context of LLM inference engine tuning, $f(\mathbf{x})$ can represent the objective functions of request throughput $T(\mathbf{x})$, request tail latency $L(\mathbf{x})$, average TTFT $\Phi(\mathbf{x})$, and average TPOT $\Theta(\mathbf{x})$.

SCOOT uses the Gaussian process (GP) as the surrogate model. Given a parameter configuration $\mathbf{x}$, GP assumes that the probability distribution of $f(\mathbf{x})$ follows a Gaussian distribution whose mean $\mu(f(\mathbf{x}))$ and the variance $\sigma^2(f(\mathbf{x}))$ are respectively computed by

$$\mu(f(\mathbf{x})) = k(\mathbf{x}, \mathbf{X})(k(\mathbf{X}, \mathbf{X}) + \tau^2 I)^{-1} \mathbf{Y}, \quad (4)$$

$$\sigma^2(f(\mathbf{x})) = k(\mathbf{x}, \mathbf{x}) - k(\mathbf{x}, \mathbf{X})(k(\mathbf{X}, \mathbf{X}) + \tau^2 I)^{-1} k(\mathbf{X}, \mathbf{x}), \quad (5)$$

where $\mathbf{X}$ represents the previously evaluated configurations, $\mathbf{Y}$ denotes the corresponding observed objective values, τ^2 is the level of white noise, and $k(\mathbf{x}, \mathbf{x}')$ is the covariance function that quantifies the similarity between input points $\mathbf{x}$ and $\mathbf{x}'$ for inferring the relationships between their objective function values. SCOOT employs the Matern kernel ($\frac{3}{2}$) with input wrapping [20] as the

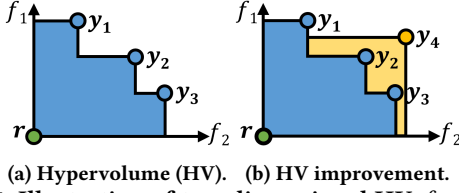


Figure 4: Illustration of two-dimensional HV. f_1 and f_2 are two objective functions and r is the reference point. (a) Blue area represents the HV of the existing solution set. (b) Yellow area depicts the HV improvement after adding y_4 .

covariance function due to its capability to balance the smoothness and flexibility when modeling unknown functions. SCOOT utilizes maximum likelihood estimation [21] to learn τ^2 during tuning.

For SOBO, a single-output GP is leveraged to predict the objective function. For MOBO, SCOOT adopts a multi-output GP by considering each output to be independent. During the tuning process, the prediction accuracy of the GP model is continuously improved as more observations are collected.

4.4 Acquisition Function

The acquisition function of BO assesses parameter configurations in the search space, which calculates a score for each configuration point $\mathbf{x}$ according to the surrogate model's predicted mean $\mu(f(\mathbf{x}))$ (indicating the expected performance) and variance $\sigma^2(f(\mathbf{x}))$ (representing uncertainty). Parameter configurations with high scores are more likely to be suggested for evaluation. Different acquisition functions balance exploration (visiting areas with high uncertainty) and exploitation (intensively searching areas with good known objective values) in different manners.

4.4.1 SOBO Acquisition Function. For SOBO, commonly used acquisition functions include upper confidence bound (UCB), probability of improvement (PI), and expected improvement (EI). UCB incorporates both mean and variance into the score and prioritizes the point with a high balance of exploration and exploitation through a trade-off parameter β , which is expressed by

$$UCB(\mathbf{x}) = \mu(f(\mathbf{x})) + \beta \cdot \sigma(f(\mathbf{x})), \quad (6)$$

where $\sigma(f(\mathbf{x})) = \sqrt{\sigma^2(f(\mathbf{x}))}$ represents the standard deviation.

PI prioritizes the point that is likely to yield an improvement over the current known best observation, which is expressed by

$$PI(\mathbf{x}) = P(f(\mathbf{x}) + \xi > f(\mathbf{x}^+)), \quad (7)$$

where $\mathbf{x}^+$ is the point that has the largest objective function value known so far, and ξ is leveraged to encourage exploration.

EI not only considers the probability of objective improvement but also the magnitude of the expected improvement, which is computed by

$$EI(\mathbf{x}) = \mathbb{E}(\max(0, f(\mathbf{x}) + \xi - f(\mathbf{x}^+))), \quad (8)$$

where $\mathbf{x}^+$ represents the best observed configuration point as well, and ξ is also a parameter used to encourage exploration.

4.4.2 MOBO Acquisition Function. For MOBO, expected hypervolume improvement (EHVI) is widely adopted as the acquisition function to identify promising parameter configurations that denote optimal trade-offs between multiple optimization objectives.

EHVI assesses a new configuration point based on the potential improvement it might bring to the hypervolume of the existing solution set, which is calculated by

$$EHVI(\mathbf{x}) = \mathbb{E}(\max(0, HV(\mathcal{Y} \cup \{f(\mathbf{x})\}) - HV(\mathcal{Y}))), \quad (9)$$

where $HV(\cdot)$ is the hypervolume function and $\mathcal{Y}$ denotes the objective values of previously evaluated configurations.

The hypervolume function $HV(\cdot)$ is calculated by measuring the volume of space enclosed between the Pareto frontier and a reference point r , where r is typically a lower bound point for all objective functions. When there are only two optimization objectives, $HV(\cdot)$ is calculated by summing the areas of rectangles formed between each point on the Pareto frontier and r . Figure 4 presents how the two-dimensional $HV(\cdot)$ is computed.

A larger $HV(\cdot)$ value indicates a Pareto front that covers a larger objective space, offering a broader range of promising solutions. By maximizing the EVHI, the search process can be guided towards configurations that can improve the overall quality and diversity of the Pareto frontier, thus optimizing multiple objectives simultaneously and providing more optional configurations for customers.

4.5 Configuration Suggestion

4.5.1 SOBO Suggestion. For SOBO, a common practice is to select a specific acquisition function and suggest parameter configurations that maximize the acquisition function. However, when it comes to real-world applications, the selected acquisition function might work sub-optimally for the tuning task, and the best acquisition function is challenging to identify beforehand [22].

To mitigate this issue, SCOOT employs multi-objective acquisition function ensemble (MACE) [23]. MACE uses three acquisition functions, UCB, PI, and EI, to assess configuration points and runs a solver to find out the Pareto frontier that denotes the optimal trade-offs of the three acquisition functions. Suggested configurations are randomly selected in the Pareto frontier. With MACE, SCOOT can avoid using sub-optimal acquisition functions all the time, thereby improving the quality of suggested parameter configurations.

During the tuning process, for UCB, β is dynamically adjusted, and in the t th tuning iteration, β is calculated by $2\log(\frac{t^2\pi^2}{6\delta})$, where $\delta = \frac{1}{t^2}$ [24]. For EI and PI, ξ is set to a fixed value of 0.0001.

4.5.2 MOBO Suggestion. For MOBO, SCOOT leverages the EHVI as the acquisition function and runs a solver to suggest parameter configurations by maximizing EHVI. The default configuration of the inference engine is chosen as the reference point r .

4.5.3 Known Constraint Assurance. For both SOBO and MOBO, in the process of solvers solving optimization problems, the solution space is pruned by known constraints.

4.6 Parallel Configuration Suggestion

With the popularity of LLMs, an increasing number of LLM inference services are being deployed, and tuning them incurs substantial time overhead. To mitigate this problem, SCOOT suggests multiple parameter configurations at a time and utilizes multiple sets of computing resources to conduct stress testing in parallel.

Given the parallelism degree k , for SOBO, SCOOT randomly selects k points from the Pareto frontier of the three acquisition

Algorithm 1: SCOOT: BO-based Performance Tuning

Input: N : total observation number; k : parallelism degree;
 // Initialize observations
 1 $O \leftarrow \text{UNIFORM_SAMPLE_AND_EVALUATE}()$
 2 $RF \leftarrow \text{TRAIN}(O)$ // Train random forest regression
 3 $\Delta \leftarrow 0.5$; $c \leftarrow 0$; $v \leftarrow 0.05$
 4 **while** $|O| < N$ **do**
 // Suggest k configurations
 5 $\{x_1, \dots, x_k\} \leftarrow \text{SUGGEST}(k, O, RF, \Delta)$
 // Collect performance metrics
 6 $\{y_1, \dots, y_k\} \leftarrow \text{EVALUATE}(\{x_1, \dots, x_k\})$
 // Adjust feasibility threshold
 7 **if** there are invalid y in $\{y_1, \dots, y_k\}$ **then**
 8 $\Delta \leftarrow \min(0.75, \max(0.5, \Delta + v))$
 9 $c \leftarrow 0$ // Clear feasible suggestion count
 10 **else**
 11 $c \leftarrow c + k$
 12 **if** $c \geq 5$ **then**
 13 $\Delta \leftarrow \max(0.25, \Delta - v)$
 14 $c \leftarrow c - 5$
 // Update observations
 15 $O \leftarrow \text{UNION}(O, \{x_1, \dots, x_k\}, \{y_1, \dots, y_k\})$
 16 $RF \leftarrow \text{TRAIN}(O)$ // Refine Random Forest

functions for suggesting. For MOBO, the top- k points with the largest EHVI are suggested. Thus, tuning can be accelerated by a factor of k when the total observation number is fixed.

Although the parallel suggestion requires additional computing resources to evaluate configurations in parallel, many computing resources in production clusters are often idle during off-peak hours such as nights and weekends, which can be conveniently used for tuning LLM inference engines.

4.7 Random Forest-based POF Learning

To handle hidden constraints, a common practice is to assign penalty objective values to the infeasible parameter configurations that cause engine crashes. However, it is challenging to set appropriate penalty objective values. Besides, penalty objective values often hurt the surrogate model [25]. Therefore, SCOOT leverages random forest regression to learn the $POF(\cdot)$ and handle hidden constraints by restricting the solver to adhere to Eq. (3) in the process of configuration suggestion. In this way, SCOOT can substantially reduce invalid observations without affecting the surrogate model.

For the hidden constraint expressed in Eq. (3), the feasibility probability threshold Δ is critical. If Δ is fixed, setting it too high may result in repeatedly suggesting configuration points near already observed feasible configurations, potentially missing out on other superior configuration points. Conversely, setting Δ too low may lead to suggestions of infeasible configurations. Hence, SCOOT dynamically adjusts the threshold Δ during the tuning process.

The dynamic adjustment process of Δ can be found in Algorithm 1, where we can observe that when infeasible parameter configurations are suggested, Δ is increased to reduce the likelihood of further suggesting infeasible configurations. Besides, after continuously suggesting feasible configurations five times, Δ is decreased to

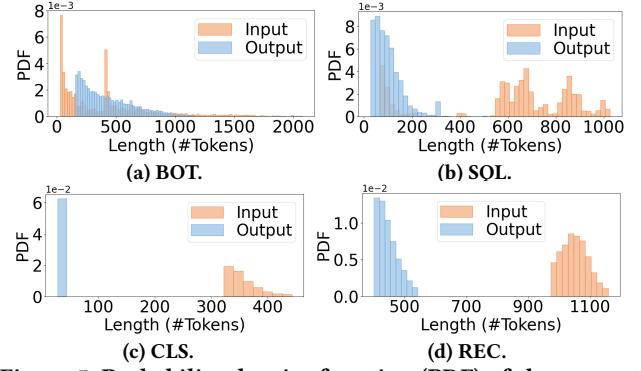


Figure 5: Probability density function (PDF) of the request input and output lengths for four application request traces.

allow the discovery of potential superior configurations. Additionally, during the tuning process, the random forest is continuously refined using the latest observations to enhance prediction accuracy, ensuring that hidden constraints are intelligently followed without compromising the quality of performance tuning.

Since known and hidden constraints pure the solution space, solvers may be unable to give a sufficient number of solutions during the configuration suggestion. In such scenarios, SCOOT randomly samples configuration points with Sobol sequence-based Quasi-Monte Carlo to make up for the shortfall.

4.8 Ensuring SLO Robustness

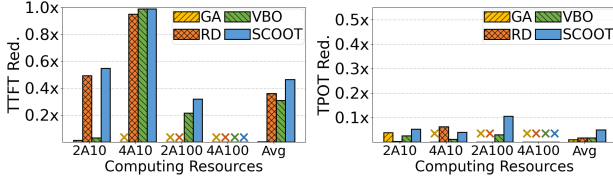
Through extensive experiments, we find that when the performance tuning is conducted under different request arrival orders, the best parameter configurations are almost the same, but the corresponding optimized SLOs vary. Thus, after performance tuning, we conduct stress testing multiple times again using the best configuration, varying request arrival orders, and select the worst-case objectives as the SLOs to ensure SLO Robustness. Finally, we summarize the overall tuning procedure of SCOOT in Algorithm 1.

5 Experimental Evaluation

5.1 Experiment Setup

We implement SCOOT on top of HEBO [22], where the random forest regression is implemented using the sklearn [26] library. Request traces are collected from four LLM inference services at Ant Group, including applications of text-to-SQL (SQL), chatbot (BOT), classification (CLS), and recommendation (REC). Since the computational load of an LLM inference request depends on its input and output lengths, to enhance the robustness of optimized SLOs, SCOOT uses requests with the longest 50% of output lengths for stress testing. The input and output lengths of these requests are presented in Fig. 5. For SQL and BOT, TTFT and TPOT are optimization objectives at the same time. For CLS and REC, 95th percentile latency and request throughput are utilized as optimization objectives, respectively.

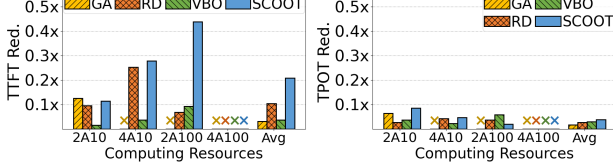
Multiple sets of computing resources are utilized in experiments, including 2A10, 4A10, 2A100, and 4A100, where A10 represents the NVIDIA A10 24GB GPU, A100 denotes the NVIDIA A100 80GB GPU, and the integer before the GPU type indicate the number of GPUs. For each set of computing resources, 256GB CPU memory and one 2.90GHz 32-core Intel(R) Xeon(R) CPU are equipped. Besides, A100 GPUs are connected over NVLink while A10 GPUs are



(a) TTFT reduction.

(b) TPOT reduction.

Figure 6: SLO optimization for BOT under LLAMA2-7B.



(a) TTFT reduction.

(b) TPOT reduction.

Figure 7: SLO optimization for BOT under LLAMA2-13B.

connected over PCIe. We use LLAMA2-7B and LLAMA2-13B [27] in experiments because the model architecture of LLAMA is representative and fine-tuned 7B and 13B LLMs can fulfill the requirements for most applications in practice. Due to the high usability and popularity of vLLM, we adopt vLLM-0.4.2 as the inference engine to tune and provide its known constraints in Appendix A.2.

In each configuration evaluation, when the inference engine is started with the suggested configuration, we send requests from the request trace for 100 seconds and the request arrival times are generated using Poisson distribution with various request rates. For A10, the request rates for SQL, BOT, CLS, and REC are 5, 5, 10, and 15, respectively. For A100, the request rates of these request traces are twice that of A10. The request rate is set according to the actual workload of the applications as well as considering the computing capability and GPU memory capacity of GPUs.

We compare SCOOT with three baselines, including random sampling (RD), genetic algorithm (GA), and Vanilla BO (VBO). Detailed baseline description is provided in Appendix A.3.

5.2 SLO Optimization

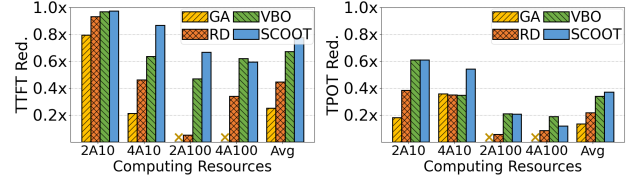
In the experiments of SLO optimization, we limit the total observation number to 30 and set the suggestion parallelism degree (PD) of SCOOT to one. The experimental results are presented in Figures 6, 7, 8, 9, 10, and 11, where we not only present the SLO improvement under each set of computing resources but also the average SLO improvement across various computing resources. The bar represents the SLO improvement compared to the default parameter configuration, and **the higher the bars, the better**. In addition, × represents that no better configuration than the default configuration is found.

For BOT and SQL applications, TTFT and TPOT are optimized simultaneously, and a Pareto frontier is resolved. We choose SLOs of the configuration with maximal HV from the Pareto frontier as the tuning result to report since it represents the best trade-off between TTFT and TPOT. HV of a single configuration point x is calculated by

$$HV(x) = \max(0, y_x^\phi - y_r^\phi) \cdot \max(0, y_x^\theta - y_r^\theta), \quad (10)$$

where y_x^ϕ , y_x^θ , y_r^ϕ , and y_r^θ denote the observed TTFT and TPOT of x and the reference point r , respectively.

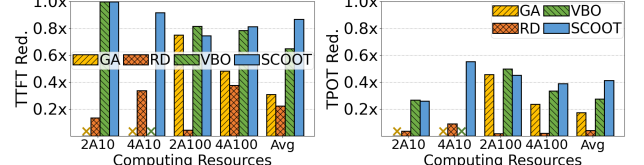
Figures 6, 7, 8, and 9 show that compared to the default configuration and baselines, SCOOT decreases the TTFT and TPOT



(a) TTFT reduction.

(b) TPOT reduction.

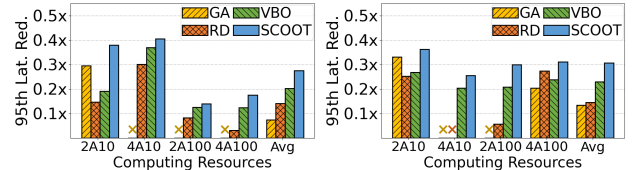
Figure 8: SLO optimization for SQL under LLAMA2-7B.



(a) TTFT reduction.

(b) TPOT reduction.

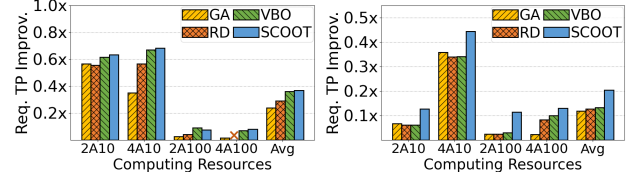
Figure 9: SLO optimization for SQL under LLAMA2-13B.



(a) LLAMA2-7B.

(b) LLAMA2-13B.

Figure 10: Tail latency reduction for CLS.



(a) LLAMA2-7B.

(b) LLAMA2-13B.

Figure 11: Request TP improvement for REC.

by up to 98.9% and 10.5% for the BOT application, respectively, and reduces the TTFT and TPOT by up to 99.8% and 61.0% for the SQL application, separately. In addition, SCOOT surpasses all the baselines in terms of average TTFT and TPOT reduction across various computing resources for BOT and SQL applications, which confirms SCOOT's superiority in multi-objective optimization.

Figures 10 and 11 present that compared to the default configuration and baselines, SCOOT reduces request tail latency by up to 40.6% for the CLS application and increases request throughput by up to 68.3% for the REC application. Besides, SCOOT outperforms all the baselines in terms of average tail latency reduction and request throughput improvement across various computing resources for CLS and REC applications, respectively, which confirms SCOOT's superiority in single-objective optimization.

While in rare situations, SCOOT failed to outperform certain baselines in SLO optimization, SCOOT's tuning performance is more consistent. As shown in Fig. 9, in the case of 2A100, VBO slightly outperforms SCOOT in both TTFT and TPOT reduction. However, in the case of 4A10, VBO fails to find a configuration that is better than the default configuration, while SCOOT greatly reduces TTFT and TPOT. Such a consistently superior performance makes SCOOT outperform all the baselines in the average performance of SLO optimization for various LLMs and applications.

GA optimizes around the initially sampled configurations. With a limited number of observations, both the population size and the

number of iterations of GA are restricted to a small value. Thus, GA not only lacks diversity in solutions but also cannot fully utilize historical information to guide the search process, and GA can hardly tackle constraints, hence often performing poorly.

RD is unable to utilize historical information to guide the sampling process and can not handle constraints. However, due to the uniform sampling, RD can explore a large range in the search space and hence has a higher diversity in sampled configurations than GA. Hence, RD often performs better than GA.

By leveraging BO to intelligently utilize historical observations to guide the tuning process, VBO often outperforms RD and GA in SLO optimization. However, by assigning a penalty objective value to unfeasible configurations, the surrogate model will confuse points near infeasible configurations with points that are feasible but perform poorly. This can lead to a lack of subsequent exploration in the adjacent area of infeasible configurations, thus missing out on some potentially superior solutions.

SCOOT's outstanding performance primarily comes from the employment of BO, which efficiently resolves black-box optimization problems, and the random forest-based POF learning, which intelligently handles hidden constraints without affecting the GP-based surrogate model. Besides, by dynamically adjusting the POF threshold Δ , SCOOT can explore the region near the infeasible configurations after continuous exploration of feasible configurations so as to fully explore the solution space.

5.3 Tuning Efficiency

We conduct experiments to tune the CSL and BOT applications under 2A100 with different parallelism degrees (PDs). In the experiments, the number of observations is also limited to 30 when PD is set to 2. Figures 12a and 12b show the SCOOT's performance in SLO optimization under various PDs, while Fig. 12c presents the total tuning time under various PDs.

Figures 12a and 12b show that when PD is set to 2, using the parallel suggestion achieves the same performance in SLO optimization as not using it. Besides, as presented in Fig. 12c, when PD is set to 2, the parallel suggestion technique significantly reduces the total tuning time, nearly cutting it in half, which demonstrates that the parallel suggestion can effectively accelerate the tuning process by PD times. Furthermore, Figure 12c shows that compared with the time of configuration evaluation (i.e., stress testing), the time required for configuration suggestion is negligible, which validates the efficiency of applying BO to tune LLM inference engines.

We also increase PD and conduct experiments. However, we find that when PD is set to a large value, such as greater than 4, SCOOT's performance of SLO optimization is compromised, and increasing the total observation number can mitigate this issue. In the future, we will explore the best number of observations under different PDs to find the most cost-effective acceleration solution.

5.4 Supplementary Experiments

In the appendix, we evaluate the effectiveness of the random forest (RF)-based learning of hidden constraints through ablation studies, confirm the superiority of SCOOT in both the optimality and diversity of SLO optimization in multi-objective situations, and show that SCOOT can outperform baselines and human experts through real-world inference service optimization results, where both vLLM and TensorRT-LLM are leveraged to verify the universality of SCOOT.



(a) Tail latency reduction for LLMs (b) TTFT & TPOT reduction for LLMs (c) Performance tuning time for CLS and BOT under various PDs

Figure 12: Tuning efficiency of the parallel suggestion.

6 Related Works

6.1 Advanced LLM Inference Techniques

Existing inference engines such as vLLM and TensorRT-LLM support a variety of advanced inference techniques to improve the speed and throughput of LLM inference, including a series of kernel-level optimizations such as continuous batching [6], paged attention [7], flash attention [28, 29], and chunked prefill [8]. Besides, prefix caching [30] is employed by inference engines to reuse computed key and value tensors of requests' common prefix texts to reduce redundant memory allocation and computation for newly arrived requests, which can significantly improve the serving efficiency when requests share a long common prefix. Moreover, speculative decoding [18, 31–34] is also adopted by inference engines to accelerate LLM inference, where an efficient draft model is leveraged to generate tokens, and the LLM occasionally refines the draft.

Some of these technologies are effective in specific scenarios and have been implemented as configurable parameters of inference engines. However, how to combine these technologies to fully exploit the capabilities of inference engines has not been adequately studied, a gap that this study aims to fill.

6.2 Automatic Performance Tuning Approaches

Performance tuning techniques have been widely applied across a wide variety of fields, including database tuning [35–37], Spark configuration tuning [38–41], compiler optimization [25], and tuning of web-relevant applications [42, 43]. The vast majority of these performance tuning studies adopt BO to find optimized parameter configurations since BO is theoretically grounded and can efficiently learn the relationship between performance and parameters from evaluations, thus intelligently tuning parameters for performance improvement. Because of these advantages, BO is also employed in resource allocation studies [44–46].

To the best of our knowledge, previous studies have not focused on tuning LLM inference engines and can not address all the three unique challenges faced in inference engine tuning.

7 Conclusion

In this paper, we propose SCOOT to optimize SLOs for LLM inference services via BO-based automatic inference engine tuning. SCOOT uses RF to learn hidden constraints and employs the parallel suggestion to speed up tuning. Extensive experiments are conducted to confirm both the effectiveness and efficiency of SCOOT.

Acknowledgments

This work is supported in part by the Postgraduate Research & Practice Innovation Program of Jiangsu Province (KYCX24_0247), the Nanjing Key S&T Special Projects (202309006), and the Ant Research Program of Ant Group.

References

- [1] Bonan Min, Hayley Ross, Elior Sulem, et al. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40, 2023.
- [2] Alibaba cloud bailian platform. <https://www.aliyun.com/product/bailian>, 2024.
- [3] Aws sagemaker. <https://aws.amazon.com/sagemaker>, 2024.
- [4] vllm. <https://github.com/vllm-project/vllm>, 2024.
- [5] Tensorrt-llm. <https://github.com/NVIDIA/TensorRT-LLM>, 2024.
- [6] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, et al. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [8] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, et al. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024.
- [9] Wenchao Xu, Jinyu Chen, Peirong Zheng, et al. Deploying foundation model powered agent services: A survey. 2024.
- [10] Herodotos Herodotou, Yuxing Chen, and Jiaheng Lu. A survey on automatic parameter tuning for big data processing systems. *ACM Computing Surveys (CSUR)*, 53(2):1–37, 2020.
- [11] Tito Homem-de Mello and Güzin Bayraksan. Monte carlo sampling-based methods for stochastic optimization. *Surveys in Operations Research and Management Science*, 19(1):56–85, 2014.
- [12] Sourabh Katoch, Sumit Singh Chauhan, and Vijay Kumar. A review on genetic algorithm: past, present, and future. *Multimedia tools and applications*, 80:8091–8126, 2021.
- [13] Sindhu Padakandla. A survey of reinforcement learning algorithms for dynamically varying environments. *ACM Computing Surveys (CSUR)*, 54(6):1–25, 2021.
- [14] Xilu Wang, Yaochu Jin, Sebastian Schmitt, and Markus Olhofer. Recent advances in bayesian optimization. *ACM Computing Surveys*, 55(138):1–36, 2023.
- [15] Nazan Khan, David E Goldberg, and Martin Pelikan. Multi-objective bayesian optimization algorithm. In *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation*, pages 684–684, 2002.
- [16] Kaifeng Yang, Michael Emmerich, André Deutz, and Thomas Bäck. Multi-objective bayesian global optimization using expected hypervolume improvement gradient. *Swarm and evolutionary computation*, 44:945–956, 2019.
- [17] Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization. *Advances in Neural Information Processing Systems*, 33:9851–9864, 2020.
- [18] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
- [19] Russel E Caflisch. Monte carlo and quasi-monte carlo methods. *Acta numerica*, 7:1–49, 1998.
- [20] Jasper Snoek, Kevin Swersky, Rich Zemel, and Ryan Adams. Input warping for bayesian optimization of non-stationary functions. In *International conference on machine learning*, pages 1674–1682. PMLR, 2014.
- [21] Matthias Seeger. Gaussian processes for machine learning. *International journal of neural systems*, 14(02):69–106, 2004.
- [22] Alexander I Cowen-Rivers, Wenlong Lyu, Rasul Tutunov, et al. Hebo: Pushing the limits of sample-efficient hyper-parameter optimisation. *Journal of Artificial Intelligence Research*, 74:1269–1349, 2022.
- [23] Shuhan Zhang, Fan Yang, Changhao Yan, et al. An efficient batch-constrained bayesian optimization approach for analog circuit synthesis via multiobjective acquisition ensemble. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(1):1–14, 2021.
- [24] Niranjan Srinivas, Andreas Krause, Sham Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: no regret and experimental design. In *Proceedings of the 27th International Conference on Machine Learning*, pages 1015–1022, 2010.
- [25] Erik Orm Hellsten, Artur Souza, Johannes Lenfers, et al. Baco: A fast and portable bayesian compiler optimization framework. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, pages 19–42, 2023.
- [26] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [27] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. 2023.
- [28] Tri Dao, Dan Fu, Stefano Ermon, et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [29] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2023.
- [30] Nikhil Jha and Kevin Wang. Improving large language model throughput with efficient long-term memory management.
- [31] Heming Xia, Tao Ge, Peiyi Wang, et al. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3909–3925, 2023.
- [32] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024.
- [33] Sehoon Kim, Kartikeya Mangalam, Suhong Moon, et al. Speculative decoding with big little decoder. *Advances in Neural Information Processing Systems*, 36, 2024.
- [34] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, et al. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36, 2024.
- [35] Xinyi Zhang, Hong Wu, Zhuo Chang, et al. Restune: Resource oriented tuning boosted by meta-learning for cloud databases. In *Proceedings of the 2021 international conference on management of data*, pages 2102–2114, 2021.
- [36] Jiale Lao, Yibo Wang, Yufei Li, et al. Gptuner: A manual-reading database tuning system via gpt-guided bayesian optimization. 2023.
- [37] Tiannuo Yang, Wen Hu, Wangqi Peng, et al. Vdtuner: Automated performance tuning for vector data management systems. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 4357–4369, 2024.
- [38] Anastasios Gounaris, Georgia Kougka, and Ruben others Tous. Dynamic configuration of partitioning in spark applications. *IEEE Transactions on Parallel and Distributed Systems*, 28(7):1891–1904, 2017.
- [39] Ayat Fekry, Lucian Carata, Thomas Pasquier, et al. To tune or not to tune? in search of optimal configurations for data analytics. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2494–2504, 2020.
- [40] Yang Li, Huaijun Jiang, Yu Shen, et al. Towards general and efficient online tuning for spark. *Proceedings of the VLDB Endowment*, 16(12):3570–3583, 2023.
- [41] Yu Shen, Xinyuyang Ren, Yupeng Lu, Huaijun Jiang, Huangyong Xu, Di Peng, Yang Li, Wentao Zhang, and Bin Cui. Rover: An online spark sql tuning service via generalized transfer learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4800–4812, 2023.
- [42] Valentin Dalibard, Michael Schaarschmidt, and Eiko Yoneki. Boat: Building auto-tuners with structured bayesian optimization. In *Proceedings of the 26th International Conference on World Wide Web*, pages 479–488, 2017.
- [43] Dan Li and Evangelos Kanoulas. Bayesian optimization for optimizing retrieval systems. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, pages 360–368, 2018.
- [44] Tirthak Patel and Devesh Tiwari. Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 193–206. IEEE, 2020.
- [45] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Satori: efficient and fair resource partitioning by sacrificing short-term benefits for long-term gains. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 292–305. IEEE, 2021.
- [46] Tiannuo Yang, Ruobing Chen, Yusen Li, Xiaoguang Liu, and Gang Wang. Cotuner: A hierarchical learning framework for coordinately optimizing resource partitioning and parameter tuning. In *Proceedings of the 52nd International Conference on Parallel Processing*, pages 317–326, 2023.
- [47] Marc Stelmack, Nari Nakashima, and Stephen Batill. Genetic algorithms for mixed discrete/continuous optimization in multidisciplinary design. In *7th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization*, page 4771, 1998.
- [48] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.
- [49] llama.cpp: Llm inference in c++. <https://github.com/ggerganov/llama.cpp>, 2024.
- [50] vllm performance and tuning. <https://docs.vllm.ai/en/v0.4.2/models/performance.html>, 2024.
- [51] Best practices for tuning the performance of tensorrt-llm. <https://github.com/NVIDIA/TensorRT-LLM/blob/main/docs/source/performance/perf-best-practices.md>, 2024.

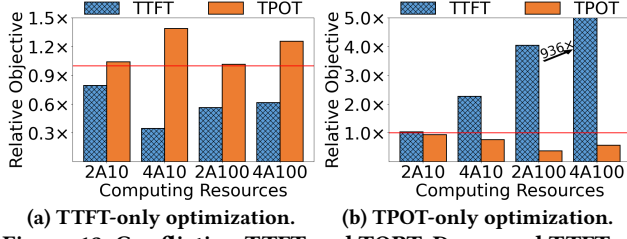


Figure 13: Conflicting TTFT and TPOT. Decreased TTFT often leads to increased TPOT, and vice versa. The red line indicates the objective value under the default parameter configuration.

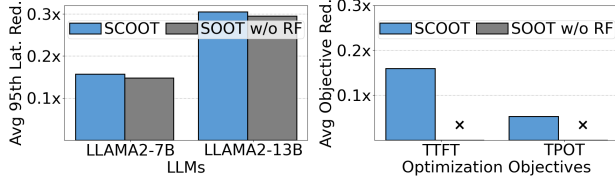


Figure 14: Effectiveness of random forest-based POF learning.

A Appendix

A.1 Conflicting TTFT and TPOT

We separately take TTFT and TPOT as the only optimization objectives and conduct grid searches for inference services of LLAMA2-13B deployed on various computing resources under the BOT request trace. Experimental results are shown in Fig. 13, which confirms that for TTFT and TPOT, optimizing for one always compromised the other. The reason is that, to decrease TPOT, it is significant to decrease the batch size controlled by the parameter max-num-seqs to reduce the per-token latency, which causes a long queuing time for requests, thus leading to a large TTFT. Conversely, to reduce TTFT, a large batch size is always set to allow newly arrived requests to be processed promptly without queuing for a long time. However, a large batch size often slows down inference, leading to increased TPOT.

A.2 Known Constraints of Inference Engines

A.2.1 vLLM. For vLLM-0.4.2 and vLLM-0.5.5, the known constraints include that max-num-batched-tokens must be greater than or equal to max_num_seqs, the enable-chunked-prefill and enable-prefix-caching cannot be set to True simultaneously, and when the enable-chunked-prefill is not enabled, max-num-batched-tokens needs to be greater than or equal to the max model length specified in the model configuration file.

A.2.2 TensorRT-LLM. For TensorRT-LLM-0.15.5, the known constraints include that max-num-tokens must be greater than or equal to max_batch_size, use-paged-context-fmha must be set to True if enable-chunked-context is enabled, and when the enable-chunked-context is not enabled, max-num-tokens needs to be greater than or equal to the max model length specified in the model configuration file.

A.3 Baseline Description

- **Random Sampling (RD):** Sobol sequence-based Quasi-Monte Carlo [19] is utilized to uniformly sample configuration points from the search space.

- **Genetic Algorithm (GA):** Mixed variable GA [47] is used to resolve single-objective optimization problems while NSGA2 [48] is utilized to handle multi-objective situations. For GA, we assign a penalty objective value when constraints are violated and set the population size to 10.
- **Vanilla BO (VBO):** VBO uses UCB and EHVI as acquisition functions to handle single-objective and multi-objective optimization situations, respectively. For VBO, we assign a penalty objective value when constraints are violated and use a random forest-based surrogate model to learn objective functions and constraints at the same time.

All of these approaches are developed using HEB0 [22], which has highly optimized implementations. Therefore, we think they are competitive and suitable as baselines.

A.4 Ablation Study Experiments

We conduct experiments to confirm the effectiveness of the random forest (RF)-based learning of hidden constraints under 2A100 and 4A100. The average SLO optimization results across various computing resources are presented in Fig. 14. Since requests from the CLS application seldom cause engine crashes, not applying RF-based POF learning has little impact on SCOOT's performance. However, since BOT application requests cause many hidden constraints, RF-based POF learning can greatly enhance SLO optimization.

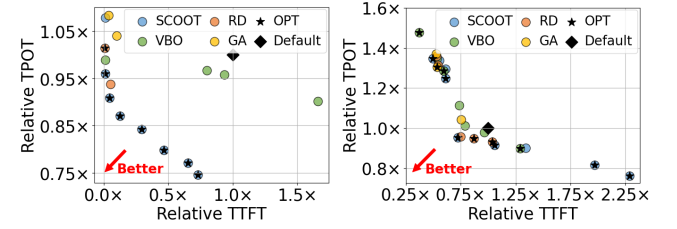


Figure 15: Solution quality for BOT.

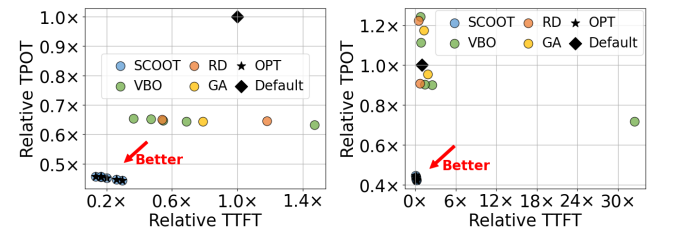


Figure 16: Solution quality for SQL.

A.5 Solution Quality Experiments

For multi-objective optimization, we evaluate the quality of solutions from two perspectives. First, the optimality of solutions, that is, whether the optimized parameter configurations lie on the Pareto frontier of TTFT and TPOT. Second, we focus on the diversity of solutions, that is, whether the range of optimized parameter configurations on the Pareto frontier is sufficiently broad to offer customers promising optional configurations to select based on their TTFT and TPOT requirements.

Figures 15 and 16 plot the Pareto sets of configuration points sampled in the process of tuning the BOT and SQL applications under 4A10 for SCOOT and baselines. The plotted points represent the “locally optimal solutions” identified by each tuning method. Additionally, we mark the global Pareto set from these locally optimal configuration points with ★. From Fig. 15, we can observe that for BOT, the majority of globally optimal configuration points are contributed by SCOOT. Besides, SCOOT yields a wider range of its globally optimal points compared with baselines. From Fig. 16, we can find that for SQL, all the globally optimal configuration points are contributed by SCOOT. These experimental results confirm the superiority of SCOOT in both the optimality and diversity of SLO optimization in multi-objective situations.

SCOOT achieves a broader range of solutions because it uses EHVI as the acquisition function for multi-objective problems to optimize the Pareto front of TTFT and TPOT. Besides, since the default configuration is chosen as the reference point, EHVI ensures that the TTFT and TPOT of the configuration with maximal HV are both better than the default, achieving the goal of optimizing both objectives simultaneously. Since VBO also employs EHVI, its solution range is broad. However, because it cannot effectively handle constraints, the optimality of the solutions is compromised.

A.6 Real-World Performance Experiments

Two real-world LLM inference services at Ant Group aim to maximize throughput while maintaining acceptable TTFT and TPOT. Both services are applications for short story generation, with average input lengths of 1,000 and 500 tokens, respectively, and an average output length of 300 tokens for both. We refer to these two inference services as story-teller-long (STL) and story-teller-short (STS) separately. Both services deploy an LLM with 80 billion parameters on 8 NVIDIA L20 48GB GPUs. During the online serving of STS and STL, the client uses concurrency to limit the maximum number of requests the service handles simultaneously, and we also include the concurrency as a parameter for optimization.

We adopt vLLM-0.5.5 and TensorRT-LLM-0.15.0 as inference engines, respectively, and conduct experiments to optimize throughput for these two inference services. The TensorRT-LLM parameters to be tuned are listed in Table 3. We have compared the tuning results of SCOOT with the default parameters of the inference engine, the recommended parameters from human algorithm experts at Ant Group, and VBO. Since VBO has been previously demonstrated to always outperform both GA and RD strategies in our experiments, we have not included comparisons with RD and GA. We filter out parameter configurations that do not meet the requirements for TTFT and TPOT and select the configurations with the highest throughput. The experimental results are presented in Table 4, where the throughput is measured in requests per second (RPS).

Human experts always simply enable certain techniques, which leads to sub-optimal results. Both STL and STS requests have a short common prefix. For vLLM-0.5.5, since chunked prefill and prefix caching are incompatible, human experts enable chunked prefill for STL and prefix caching for STS, respectively. For TensorRT-LLM 0.15.0, human experts enable both chunked prefill and prefix caching for STL and STS. However, due to the lack of sophisticated tuning of the client concurrency and parameters such as tokens-per-block and max-batch-size, the throughput improvement is always limited. From the experimental results, we can see

that SCOOT consistently improves the throughput of two real-world inference services under various inference engines and significantly outperforms all baselines, which confirms the universality and practicality of SCOOT.

Table 3: TensorRT-LLM PARAMETERS TO TUNE.

Configuration Parameter	Type	Range
tp-size	Int	[1, #GPUs]
max-batch-size	Int	[64, 8192]
max-num-tokens	Int	[64, 8192]
tokens-per-block	Enum	{4, 8, 16, 32, 64, 128}
use-paged-context-fmha	Bool	{True, False}
enable-chunked-context	Bool	{True, False}
enable-block-reuse	Bool	{True, False}
capacity-scheduler-policy	Enum	{FCFS, EQUAL}
context-chunking-policy	Enum	{MAX_UTI, NO_EVICT}

Table 4: THROUGHPUT OPTIMIZATION RESULTS FOR REAL-WORLD INFERENCE SERVICES.

Method	vLLM		TensorRT-LLM	
	STL	STS	STL	STS
Default	0.527	0.948	0.557	0.985
Expert	0.528	1.004	0.598	1.039
VBO	0.595	1.010	0.670	1.180
SCOOT	0.695	1.211	0.763	1.300

A.7 Discussion

A.7.1 Application Scope. SCOOT only requires the LLM inference engine to provide tunable parameters and an API-enabled entry point (e.g., FastAPI or OpenAI server). Since both widely-used LLM inference engines in cloud data centers (e.g., vLLM and TensorRT-LLM) and those deployed on personal devices with constraint resources (e.g., llama.cpp [49]) meet these requirements, SCOOT can be universally applied across a wide range of LLM inference engines.

A.7.2 Parameter Selection. The selection of parameters and their value ranges significantly impact the tuning results. We select the parameters that are generally applicable across most scenarios and are of primary concern to customers, as indicated in the official documentation of vLLM [50] and TensorRT-LLM [51]. Furthermore, to ensure a more comprehensive optimization process, we intentionally include broad value ranges of the configuration parameters for performance tuning.

A.7.3 Final Deployed Product. We leverage existing company infrastructure and develop supplementary modules to ensure smooth deployment of SCOOT. Specifically, we implement several additional modules around SCOOT, including a dataset generation module to extract the request traces from the logging system into our database and utilize scripts to convert them into SCOOT-compatible formats, and a configuration parser to parse user-defined tuning parameters, including parameter names and their ranges. If parameters-to-tune are not specified by users, a default set of parameters and ranges will also be made available to all users. Besides, a post-processing module is developed to store the suggested parameters in the database and present them to customers for selection after SCOOT finishes parameter tuning.